

Probabilistic Data Structures

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

Probabilistic Data Structures

Until now, all our data structures were guaranteed to give us correct answers with *100% certainty*.

- But not all data structures do!

Probabilistic data structures give answers with *uncertainty*.

Probabilistic Data Structures

Until now, all our data structures were guaranteed to give us correct answers with *100% certainty*.

- But not all data structures do!

Probabilistic data structures give answers with *uncertainty*.

Today: Bloom filters

- Sets where checking membership is probabilistic
 - ▶ $\text{mem?}(x) \rightarrow \text{False}$: x is *definitely not* in the set
 - ▶ $\text{mem?}(x) \rightarrow \text{True}$: x is *maybe* in the set
 - ▶ False positives are possible! But not false negatives.
- We lose certainty: sometimes we won't know for sure
- In return, we get a very space-efficient data structure!

Motivating Example: Web Malware

The Internet is a dangerous place

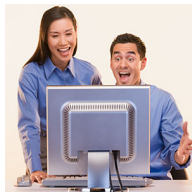
- Lots of web sites trying to infect you with malware
- Thankfully, your browser tries to protect you
 - ▶ Checks whether you're accessing a known bad site (URL)
 - ▶ Blocks your request if you do



Motivating Example: Web Malware

The Internet is a dangerous place

- Lots of web sites trying to infect you with malware
- Thankfully, your browser tries to protect you
 - ▶ Checks whether you're accessing a known bad site (URL)
 - ▶ Blocks your request if you do



accept



Motivating Example: Web Malware

The Internet is a dangerous place

- Lots of web sites trying to infect you with malware
- Thankfully, your browser tries to protect you
 - ▶ Checks whether you're accessing a known bad site (URL)
 - ▶ Blocks your request if you do



Motivating Example: Web Malware

The Internet is a dangerous place

- Lots of web sites trying to infect you with malware
- Thankfully, your browser tries to protect you
 - ▶ Checks whether you're accessing a known bad site (URL)
 - ▶ Blocks your request if you do



How does Firefox know a URL is dangerous?

- It stores dangerous URLs in a big set!
- Accept/reject = set membership!

How do we build a set?

We can think of a set as a dictionary where values don't matter.

- Only key presence does!

Set:

```
itsatrap.biz,  
freestuff.party,  
virusbasket.com.exe.tk,  
qualitywarez.xyz
```

Dictionary:

```
itsatrap.biz,      : True  
freestuff.party,   : True  
virusbasket.com.exe.tk, : True  
qualitywarez.xyz,  : True
```


How do we build a set?

We can think of a set as a dictionary where values don't matter.

- Only key presence does!

Set:

```
itsatrap.biz,  
freestuff.party,  
virusbasket.com.exe.tk,  
qualitywarez.xyz
```

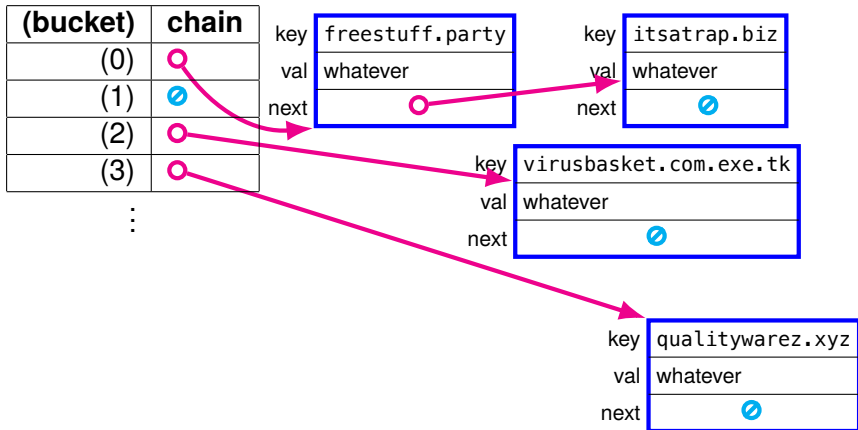
Dictionary:

```
itsatrap.biz,      : True  
freestuff.party,   : True  
virusbasket.com.exe.tk, : True  
qualitywarez.xyz,  : True
```

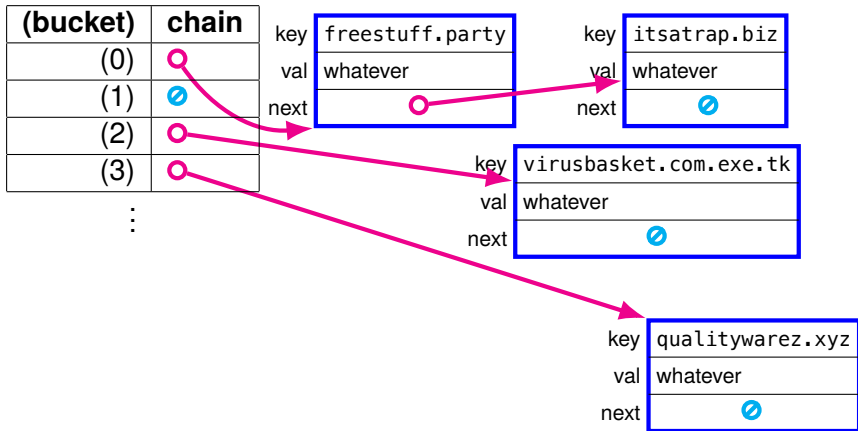
Therefore, any dictionary DS can be used as a set DS!

Let's try our go-to dictionary: hash tables!

Hash Table Sets



Hash Table Sets



Cool, so how much space are we going to need?

Napkin Math

How much space do we need for that set?

Let's make some (very conservative) guesses.

And pick concrete numbers, courtesy of Comp Sci 213.

Napkin Math

How much space do we need for that set?

Let's make some (very conservative) guesses.

And pick concrete numbers, courtesy of Comp Sci 213.

- 10 million malware sites $\rightarrow$ 10M element set

Napkin Math

How much space do we need for that set?

Let's make some (very conservative) guesses.

And pick concrete numbers, courtesy of Comp Sci 213.

- 10 million malware sites $\rightarrow$ 10M element set
- Each site (URL) is 100 characters $\rightarrow$ 100 bytes per key
- Values are booleans $\rightarrow$ 1 bit (i.e., $\frac{1}{8}$ byte) per value

Napkin Math

How much space do we need for that set?

Let's make some (very conservative) guesses.

And pick concrete numbers, courtesy of Comp Sci 213.

- 10 million malware sites $\rightarrow$ 10M element set
- Each site (URL) is 100 characters $\rightarrow$ 100 bytes per key
- Values are booleans $\rightarrow$ 1 bit (i.e., $\frac{1}{8}$ byte) per value
- Each bucket is an arrow $\rightarrow$ 8 bytes per bucket
- Suppose a load factor of 1 $\rightarrow$ 10M buckets

Napkin Math

How much space do we need for that set?

Let's make some (very conservative) guesses.

And pick concrete numbers, courtesy of Comp Sci 213.

- 10 million malware sites $\rightarrow$ 10M element set
- Each site (URL) is 100 characters $\rightarrow$ 100 bytes per key
- Values are booleans $\rightarrow$ 1 bit (i.e., $\frac{1}{8}$ byte) per value
- Each bucket is an arrow $\rightarrow$ 8 bytes per bucket
- Suppose a load factor of 1 $\rightarrow$ 10M buckets

Total: $10\text{M} \times (100 \text{ bytes} + 1 \text{ bit} + 8 \text{ bytes}) \approx 1 \text{ GB!}$

We're all set, right?

The full Firefox installer (circa FF 100) is 121 MB.

With (very) conservative assumptions, full set is $\approx$ 1GB.

- I.e., about 8 times the whole browser
- I.e., way too big to ship with it.

Oops.

We're all set, right?

The full Firefox installer (circa FF 100) is 121 MB.

With (very) conservative assumptions, full set is $\approx$ 1GB.

- I.e., about 8 times the whole browser
- I.e., way too big to ship with it.

Oops.

What is the bulk of that space used for?

We're all set, right?

The full Firefox installer (circa FF 100) is 121 MB.

With (very) conservative assumptions, full set is $\approx$ 1GB.

- I.e., about 8 times the whole browser
- I.e., way too big to ship with it.

Oops.

What is the bulk of that space used for?

The keys!

Why did we need to store the keys again?

- When lookup lands on a bucket, need to know if the contents are the key we're looking for.
- With collisions, another key could be in the bucket instead!

Stop! Question time!

Before try to cut down on space...

Anything unclear so far?

Anything I missed?

Honey, I lost my keys

What would happen if we *didn't* store the keys?

Would cut down on space for sure!

- Not only the keys themselves, but can put booleans directly in buckets (i.e., no need for arrows)
- Would need space: $10\text{M} \times 1 \text{ bit} \approx 1.2 \text{ MB!}$
 - ▶ Much better!
 - ▶ **Spoiler:** won't quite get there, but we'll get close

Honey, I lost my keys

What would happen if we *didn't* store the keys?

Would cut down on space for sure!

- Not only the keys themselves, but can put booleans directly in buckets (i.e., no need for arrows)
- Would need space: $10\text{M} \times 1 \text{ bit} \approx 1.2 \text{ MB!}$
 - ▶ Much better!
 - ▶ **Spoiler:** won't quite get there, but we'll get close

But if we don't store keys...

- Look at a bucket, no idea what key that value was for.
→ we can't handle collisions anymore.
- Still, let's see what that gives us...

Probabilistic Hash Sets

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
						T		T							T		T						

To insert a key k in the set:

- Hash k , set the corresponding bucket i to True
- Don't store k itself, though!

To check a key k for membership in the set:

- Hash k , look up the corresponding bucket i
- If bucket i is True, then *maybe* k is in the set
 - ▶ There exists a key that hashes to i which is in the set
 - ▶ Which one? No idea. Maybe k , maybe something else.
- If bucket i is False, then *definitely* k is *not* in the set!
 - ▶ No key which hashes to i is in the set!

Probabilistic Hash Sets

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
						T		T							T		T						

That gives us a *probabilistic* data structure.

- Does not always give us the right answer!
- Can have false positives, but no false negatives!
- If it says yes, the answer is really “maybe”
- But if it says no, we **know** that the answer is no!

Probabilistic Hash Sets

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
						T		T							T		T						

- Lookup key `virusbasket.com.exe.tk`
 - ▶ $h(\text{virusbasket.com.exe.tk}) = 8$
 - ▶ Could be in the set! But we're not sure.

Probabilistic Hash Sets

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
							T		T						T		T						

- Lookup key `virusbasket.com.exe.tk`
 - ▶ $h(\text{virusbasket.com.exe.tk}) = 8$
 - ▶ Could be in the set! But we're not sure.
- Insert key `itsatrap.biz`
 - ▶ $h(\text{itsatrap.biz}) = 3$
 - ▶ Let's set that bucket to True

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
			T				T		T						T		T						

Probabilistic Hash Sets

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
							T		T						T		T						

- Lookup key `virusbasket.com.exe.tk`
 - ▶ $h(\text{virusbasket.com.exe.tk}) = 8$
 - ▶ Could be in the set! But we're not sure.
- Insert key `itsatrap.biz`
 - ▶ $h(\text{itsatrap.biz}) = 3$
 - ▶ Let's set that bucket to True

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
			T				T		T						T		T						

- Lookup key `northwestern.edu`
 - ▶ $h(\text{northwestern.edu}) = 3$
 - ▶ Also could be in the set. Uh, awkward.

False Positives

If $h(\text{itsatrap.biz}) = h(\text{northwestern.edu})$, our august institution looks like a prime destination for quality malware.

- Oops.
- We'll need a backup plan (later).

False Positives

If $h(\text{itsatrap.biz}) = h(\text{northwestern.edu})$, our august institution looks like a prime destination for quality malware.

- Oops.
- We'll need a backup plan (later).

But how often *does* this happen?

- Let's assume uniform hashing function (want that anyway)
- If # of possible URLs $\ggg$ # of blacklist URLs, false positive rate is probability of a bucket being True in the table
 - ▶ The larger the number of buckets, the lower that will be
 - ▶ For n keys in the set and m buckets, roughly n/m
- 10M blacklist items, 10% false pos. = 100Mbits = 12.5MB
 - ▶ Not bad. 90% of the time, we get the correct answer.

False Positives

If $h(\text{itsatrap.biz}) = h(\text{northwestern.edu})$, our august institution looks like a prime destination for quality malware.

- Oops.
- We'll need a backup plan (later).

But how often *does* this happen?

- Let's assume uniform hashing function (want that anyway)
- If # of possible URLs $\ggg$ # of blacklist URLs, false positive rate is probability of a bucket being True in the table
 - ▶ The larger the number of buckets, the lower that will be
 - ▶ For n keys in the set and m buckets, roughly n/m
- 10M blacklist items, 10% false pos. = 100Mbits = 12.5MB
 - ▶ Not bad. 90% of the time, we get the correct answer.
 - ▶ Still, can we do better?

Stop! Question time!

Before we move on to Bloom filters...

Anything unclear so far?

Anything I missed?

Bloom Filters

If $h(\text{itsatrap.biz}) = h(\text{northwestern.edu})$, that's an unfortunate coincidence.

- With a different hash function, might not have happened.

Bloom Filters

If $h(\text{itsatrap.biz}) = h(\text{northwestern.edu})$, that's an unfortunate coincidence.

- With a different hash function, might not have happened.

Key idea:

Instead of using just *one* hash function, let's use *k* of them!

Bloom Filters

Insertion:

- Set buckets $h_1(key)$, $h_2(key)$, $\dots$, $h_k(key)$ to True
- Some of these may already be True! That's ok.
- Some of these may be the same bucket! That's ok.

Lookup:

- If *all* of buckets $h_1(key)$, $h_2(key)$, $\dots$, $h_k(key)$ are True, then *maybe* key is in the set.
- If *any* of them are False, we know it *cannot* be in the set
- False positives are much less likely now!
 - ▶ Requires k collisions. How likely is that?

Bloom Filters

Insertion:

- Set buckets $h_1(key)$, $h_2(key)$, $\dots$, $h_k(key)$ to True
- Some of these may already be True! That's ok.
- Some of these may be the same bucket! That's ok.

Lookup:

- If *all* of buckets $h_1(key)$, $h_2(key)$, $\dots$, $h_k(key)$ are True, then *maybe* key is in the set.
- If *any* of them are False, we know it *cannot* be in the set
- False positives are much less likely now!
 - ▶ Requires k collisions. How likely is that?

“Once is happenstance. Twice is coincidence. Three times is enemy action.” – Ian Fleming

Bloom Filter Example

Let's say we have 3 hash functions.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
	T						T													T			

- Insert key `itsatrap.biz`
 - ▶ $h_0(\text{itsatrap.biz}) = 3$
 - ▶ $h_1(\text{itsatrap.biz}) = 12$
 - ▶ $h_2(\text{itsatrap.biz}) = 21$
 - ▶ Let's set these buckets to True

Bloom Filter Example

Let's say we have 3 hash functions.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
	T						T													T			

- Insert key `itsatrap.biz`
 - ▶ $h_0(\text{itsatrap.biz}) = 3$
 - ▶ $h_1(\text{itsatrap.biz}) = 12$
 - ▶ $h_2(\text{itsatrap.biz}) = 21$
 - ▶ Let's set these buckets to True

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
	T		T				T					T								T	T		

Bloom Filter Example

Let's say we have 3 hash functions.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

	T						T													T			
--	---	--	--	--	--	--	---	--	--	--	--	--	--	--	--	--	--	--	--	---	--	--	--

- Insert key `itsatrap.biz`
 - ▶ $h_0(\text{itsatrap.biz}) = 3$
 - ▶ $h_1(\text{itsatrap.biz}) = 12$
 - ▶ $h_2(\text{itsatrap.biz}) = 21$
 - ▶ Let's set these buckets to True

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

	T		T				T					T								T	T		
--	---	--	---	--	--	--	---	--	--	--	--	---	--	--	--	--	--	--	--	---	---	--	--

- Lookup key `northwestern.edu`
 - ▶ $h_0(\text{northwestern.edu}) = 3$
 - ▶ $h_1(\text{northwestern.edu}) = 0$
 - ▶ $h_2(\text{northwestern.edu}) = 7$
 - ▶ Only two of these buckets are True. Not in the set!

Bloom Filter Code

```
class BloomFilter:
    let bv: BitVec? # vector of booleans, 213 style.
    let hs: VecC

    def __init__(self, m: nat?, k: nat?):
        self.bv = BitVec(m)
        self.hs = [ make_sbox_hash() for _ in range(k) ]

    def insert(self, s: str?) -> NoneC:
        for h in self.hs:
            let index = h(s) % self.bv.len()
            self.bv[index] = True

    def mem?(self, s: str?) -> bool?:
        for h in self.hs:
            let index = h(s) % self.bv.len()
            if not self.bv[index]: return False
        return True
```

False Positives, Revisited

So, how does that affect our false positive rate?

- Assume n keys in set, m buckets
- If we hash one key with one hash function, each bucket is True with probability: $P(\text{bucket True, 1 key, 1 hash}) = \frac{1}{m}$

False Positives, Revisited

So, how does that affect our false positive rate?

- Assume n keys in set, m buckets
- If we hash one key with one hash function, each bucket is True with probability: $P(\text{bucket True, 1 key, 1 hash}) = \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, 1 hash}) = 1 - \frac{1}{m}$

False Positives, Revisited

So, how does that affect our false positive rate?

- Assume n keys in set, m buckets
- If we hash one key with one hash function, each bucket is True with probability: $P(\text{bucket True, 1 key, 1 hash}) = \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, 1 hash}) = 1 - \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, } k \text{ hashes}) = (1 - \frac{1}{m})^k$

False Positives, Revisited

So, how does that affect our false positive rate?

- Assume n keys in set, m buckets
- If we hash one key with one hash function, each bucket is True with probability: $P(\text{bucket True, 1 key, 1 hash}) = \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, 1 hash}) = 1 - \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, } k \text{ hashes}) = (1 - \frac{1}{m})^k$
- $P(1 \text{ bucket False, } n \text{ keys, } k \text{ hashes}) = (1 - \frac{1}{m})^{nk}$

False Positives, Revisited

So, how does that affect our false positive rate?

- Assume n keys in set, m buckets
- If we hash one key with one hash function, each bucket is True with probability: $P(\text{bucket True, 1 key, 1 hash}) = \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, 1 hash}) = 1 - \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, } k \text{ hashes}) = (1 - \frac{1}{m})^k$
- $P(1 \text{ bucket False, } n \text{ keys, } k \text{ hashes}) = (1 - \frac{1}{m})^{nk}$
- $P(1 \text{ bucket True, } n \text{ keys, } k \text{ hashes}) = (1 - (1 - \frac{1}{m})^{nk})$

False Positives, Revisited

So, how does that affect our false positive rate?

- Assume n keys in set, m buckets
- If we hash one key with one hash function, each bucket is True with probability: $P(\text{bucket True, 1 key, 1 hash}) = \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, 1 hash}) = 1 - \frac{1}{m}$
- $P(1 \text{ bucket False, 1 key, } k \text{ hashes}) = (1 - \frac{1}{m})^k$
- $P(1 \text{ bucket False, } n \text{ keys, } k \text{ hashes}) = (1 - \frac{1}{m})^{nk}$
- $P(1 \text{ bucket True, } n \text{ keys, } k \text{ hashes}) = (1 - (1 - \frac{1}{m})^{nk})$

When we do a lookup, we look at k buckets*

- $P(k \text{ buckets all True, } n \text{ keys, } k \text{ hashes}) = (1 - (1 - \frac{1}{m})^{nk})^k$
 - ▶ That's our false positive probability

* really $\binom{m}{k}$, but that's $\approx k$ if $m \gg k$

False Positives, Revisited, Concretely

So let's plug in some numbers.

- We had $n = 10\text{M}$ malicious URLs
- We tried $m = 100\text{M}$ buckets (12.5MB)
- Previously, with one hash function, and $10\times$ as many buckets as keys, we had a 10% false positive rate before.

Suppose we pick $k = 7$ hash functions:

- $P(\text{false positive}) = 0.8\%$!
- We get a wrong answer less than 1% of the time!

False Positives, Revisited, Concretely

So let's plug in some numbers.

- We had $n = 10\text{M}$ malicious URLs
- We tried $m = 100\text{M}$ buckets (12.5MB)
- Previously, with one hash function, and $10\times$ as many buckets as keys, we had a 10% false positive rate before.

Suppose we pick $k = 7$ hash functions:

- $P(\text{false positive}) = 0.8\%$!
- We get a wrong answer less than 1% of the time!

How did we know to pick 7?

- If we fix n and m , choose k to minimize probability formula:

$$k = \frac{m}{n} \ln 2$$

Controlling False Positive Rate

For a fixed n keys and m buckets, we can pick the best k to minimize false positive rate.

But if we also get to pick m , can actually pick false positive rate!

- Picking $m = 10n$, then picking optimal k gave us 0.8%
- In general, if we have 4.8 buckets per key, then pick optimal k , get 10%
- Then every *additional* 4.8 buckets per key (and optimal k) *divides* that by 10!
 - ▶ 9.6 buckets? 1%!
 - ▶ 14.4 buckets? 0.1%!
- So depending on the space you have, and your aversion to false positives, you get to pick!

Deletion?

You cannot delete from a Bloom filter.

- You know which buckets a given key maps to
- But can't know whether other keys map to them too
- So you can't set buckets to `False`, because that might end up removing other keys as well!

Only option:

1. Rebuild the filter from scratch,
2. starting from the original data (the full set of keys), but
3. omitting the key you wish to delete

What's your backup plan?

True → can't tell if really True or false positive

How do you build your system around that uncertainty?

- **Don't care:** maybe the exact result doesn't matter, and some small amount of errors is ok (e.g., video encoding)
- **Have a backup plan:** store full set somewhere (e.g., central server), consult it when you get True
 - ▶ Contacting the server = expensive!
 - ▶ So don't want to do it too often!

What's your backup plan?

True $\rightarrow$ can't tell if really True or false positive

How do you build your system around that uncertainty?

- **Don't care:** maybe the exact result doesn't matter, and some small amount of errors is ok (e.g., video encoding)
- **Have a backup plan:** store full set somewhere (e.g., central server), consult it when you get True
 - ▶ Contacting the server = expensive!
 - ▶ So don't want to do it too often!
- **Make the false positive probability very low**
 - ▶ No computation is ever 100% reliable!
 - ▶ What's the prob your data center gets hit by a tornado?
 - ▶ What's the prob a cosmic ray hits your computer and flips a bit in memory?
 - ▶ Yes, that's a real thing
 - ▶ More buckets $\rightarrow$ false positive less likely than those!
 - ▶ **Aside:** probabilistic algorithms

Stop! Question time!

Before we move on to efficient vectors of booleans...

Anything unclear so far?

Anything I missed?

Bit Vectors

Bit Vectors

How can we store a vector of 10M booleans in 1.125 MBytes?

- With some 213-style tricks!
- This is either a preview, or a recap

Bit Vectors

How can we store a vector of 10M booleans in 1.125 MBytes?

- With some 213-style tricks!
- This is either a preview, or a recap

Bit: the smallest unit of information and storage in a computer system. 0 or 1, True or False, etc.

- To store a boolean, only need one bit
- To store a vector of booleans, want a vector of bits!

Bit Vectors

How can we store a vector of 10M booleans in 1.125 MBytes?

- With some 213-style tricks!
- This is either a preview, or a recap

Bit: the smallest unit of information and storage in a computer system. 0 or 1, True or False, etc.

- To store a boolean, only need one bit
- To store a vector of booleans, want a vector of bits!

Problem: cannot use individual bits when programming...

- DSSL2: any data is **at least** 64 bits large. Even booleans!
- C (lower-level): smallest data is 1 **byte** large
 - ▶ 1 byte = 8 bits

Bit Vectors

Key idea: treat integers as sequences of bits! Binary!

$$47 = 32 + 8 + 4 + 2 + 1$$

Bit Vectors

Key idea: treat integers as sequences of bits! Binary!

$$\begin{aligned} 47 &= 32 + 8 + 4 + 2 + 1 \\ &= 2^5 + 2^3 + 2^2 + 2^1 + 2^0 \end{aligned}$$

Bit Vectors

Key idea: treat integers as sequences of bits! Binary!

$$47 = 32 + 8 + 4 + 2 + 1$$

$$= 2^5 + 2^3 + 2^2 + 2^1 + 2^0$$

$$= 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

Bit Vectors

Key idea: treat integers as sequences of bits! Binary!

$$\begin{aligned}47 &= 32 + 8 + 4 + 2 + 1 \\&= 2^5 + 2^3 + 2^2 + 2^1 + 2^0 \\&= 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\&= 0010 \ 1111\end{aligned}$$

Bit Vectors

Key idea: treat integers as sequences of bits! Binary!

$$\begin{aligned}47 &= 32 + 8 + 4 + 2 + 1 \\&= 2^5 + 2^3 + 2^2 + 2^1 + 2^0 \\&= 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\&= 0010 \ 1111 \\&= [F, F, T, F, T, T, T, T]\end{aligned}$$

One integer, 8 booleans!

- 1 byte integers = easy to work with in most languages
- Enough to express integers from 0 to 255!
- Can use larger ints, to work with more bools at a time
- The integers don't make sense as numbers anymore; they're just bits

Bit Vectors

Want more booleans? Have a vector of integers!

- Vector of n 8-bit integers $\rightarrow$ Vector of $8 \times n$ booleans!

Bit Vectors

Want more booleans? Have a vector of integers!

- Vector of n 8-bit integers $\rightarrow$ Vector of $8 \times n$ booleans!

0	1	2
0001 0010	0000 0000	0100 0000

- Vector of 3 8-bit integers = 24 booleans
- First array element bits are booleans $0 \dots 7$
- Second array element, $8 \dots 15$
- Third array element, $16 \dots 23$
- Within each 8-bit element, ordering bits “right to left”
 - ▶ Index 0 is 0^{th} bit (the one for 2^0) of the first array element
 - ▶ Index 1 is bit 1 (for 2^1) of the first array element, etc.

Bit Vectors

0	1	2
0001 0010	0000 0000	0100 0000

Accessing boolean at index i :

- Bit $i \bmod 8$ of
- array element $\lfloor \frac{i}{8} \rfloor$

Example: boolean #22

- Array element $\lfloor \frac{22}{8} \rfloor = 2$
- Bit $22 \bmod 8 = 6$ within it
- The bit is 1 $\rightarrow$ boolean is True!

See `bv.rkt`.

- Uses bitwise ops and masking to read/write single bits.

If we have time: playing
with our Bloom filters and
bit vectors

Next time: Persistent Data
Structures